

阶段 11：SDK 化目录结构与 Public API 设计草案

状态：Proposed

更新日期：2026-08-07

适用范围：Question Evolution Agent Harness 的 Python SDK 包装、公共 API、配置对象、返回对象、CLI 兼容层和安装发布方式

目标读者：负责把当前 Agent 项目改造成可被其他 Python 项目调用的工程师

读完后应能做的事：明确 SDK 化要改哪些文件、对外暴露哪些 API、哪些现有脚本保持兼容、如何分阶段落地和验收。

1. 结论

当前项目已经具备 SDK 化基础，但还不是 SDK。

现有形态是：

代码块

- 1 用户或脚本
- 2 -> CLI / Shell
- 3 -> AgentTask JSON
- 4 -> run_loop.sh / resume_run_loop.sh
- 5 -> 实验产物
- 6 -> Agent 观察、决策、报告

SDK 化后的目标形态是：

代码块

- 1 Python 调用方
- 2 -> QuestionEvolutionAgent / QuestionEvolutionClient
- 3 -> AgentTask / AgentConfig / RunOptions
- 4 -> 结构化 AgentRunResult
- 5 -> 实验产物、事件、报告、观察和决策引用

一句话边界：**SDK 化不是重写题目进化算法，而是把现有 Agent Harness 包成稳定、可安装、可 import、可测试、可嵌入的 Python API。**

第一版 SDK 仍可以复用 `run_loop.sh` 作为复合工具。真正去 Shell 化可以放到第二阶段，避免一次性改动过大。

2. 当前现状

当前项目已经有三块接近 SDK 内核的能力：

当前能力	现状	SDK 化处理
Agent 任务对象	已有 AgentTask，能解析目标、输入、搜索模式、预算和续跑参数。	保留为核心输入类型，对外导出。
Agent 执行入口	已有 execute_agent_task()，能执行 dry-run、run、resume、review。	包装为 QuestionEvolutionAgent 的方法。
ToolRunner	已有工具执行器，能检查环境、执行完整流水线、续跑、定位实验目录。	保留为默认执行后端，允许调用方注入自定义 runner。
Agent Run 产物	已有 plan、state、events、observation、decision、report。	通过 AgentRunResult 暴露路径和摘要。
CLI	已有 question_evolution_agent.py 命令行入口。	改成 SDK 的薄封装，保持旧命令可用。

当前主要缺口：

1. 没有 `pyproject.toml`，不能作为标准 Python 包安装。
2. 没有稳定的包名和公共导出层。
3. 配置主要依赖环境变量和 shell 默认值。
4. 返回值是普通 dict，缺少稳定类型和便捷属性。
5. 调用方不能清晰区分“SDK 配置”“任务参数”“运行选项”。
6. CLI 与 SDK 内核还放在同一个文件中，长期维护边界不清晰。

3. SDK 化目标

3.1 必须实现

第一版 SDK 必须做到：

1. 支持 `pip install -e .` 后 import。
2. 支持用 Python 代码启动新实验、续跑实验、只读复盘和 dry-run。
3. 支持调用方传入结构化配置，而不是只能写环境变量。
4. 保留现有 CLI 行为，不破坏已有脚本和测试。
5. 保留现有实验产物、manifest、断点恢复和 Agent 审计文件。
6. 公共 API 不泄露内部实现细节，例如具体 shell 参数、事件文件写法、内部 helper 函数。
7. SDK 测试不访问真实模型 API，仍使用 fake runner 或夹具。

3.2 第一版不做

第一版 SDK 不做：

1. 不重写 `run_loop.sh` 主流程。
2. 不拆解完整流水线为全部 Python 原子工具。
3. 不改变算子、路由、校验、评分和 Memory 的业务语义。
4. 不让调用方绕过 Tool Registry、Policy Guard、manifest 和 schema 校验。
5. 不把所有环境变量都开放为任意透传参数。
6. 不发布为远程服务，不引入 FastAPI、Celery、LangChain 或 LangGraph。
7. 不承诺跨语言 SDK，当前只设计 Python SDK。

4. 目标目录结构

建议采用“先包一层、后迁移内部模块”的保守结构。第一阶段尽量少移动现有文件，降低断裂风险。

4.1 第一阶段目录结构

代码块

```
1  QA_Evolution_Agent/
2      pyproject.toml
3      README.md
4      question_evolution_agent.py          # 兼容旧 CLI，内部调用 SDK
5      qa_evolution_sdk/
6          __init__.py                      # 公共导出
7          client.py                        # QuestionEvolutionAgent / Client
8          config.py                        # AgentConfig、ModelEndpointConfig、
          ScoringConfig
9          options.py                      # RunOptions、ResumeOptions、
          ReviewOptions
```

```

10     results.py                                # AgentRunResult、ArtifactRefs、
    RunStatus
11     exceptions.py                            # SDK 异常层
12     callbacks.py                            # ProgressCallback、EventCallback 协
    议
13     runners.py                              # 默认 runner 适配层
14     compat.py                               # 旧模块兼容导入和路径处理
15     agent_runtime/
16         __init__.py
17     task.py
18     planner.py
19     context.py
20     policy.py
21     tools.py
22     observer.py
23     observations.py
24     decisions.py
25     events.py
26     state.py
27     reporter.py
28     tool_registry.py
29     tests/
30         test_sdk_client.py
31         test_sdk_config.py
32         test_sdk_cli_compat.py

```

这个阶段不要求把所有根目录脚本移进包内。`qa_evolution_sdk` 只作为稳定门面，调用现有 `agent_runtime` 和现有流水线脚本。

4.2 第二阶段目录结构

当第一阶段稳定后，再逐步把内部模块整理为更标准的包结构：

代码块

```

1  QA_Evolution_Agent/
2  pyproject.toml
3  qa_evolution/
4  __init__.py
5  sdk/
6  client.py
7  config.py
8  options.py
9  results.py
10 exceptions.py
11 callbacks.py
12 agent_runtime/

```

```
13     task.py
14     planner.py
15     context.py
16     policy.py
17     tools.py
18     observer.py
19     decisions.py
20     events.py
21     state.py
22     reporter.py
23     tool_registry.py
24 pipeline/
25     io.py
26     scoring.py
27     search.py
28     branch.py
29     vertical.py
30 prompts/
31 schemas/
32 memory/
33 scripts/
34     question_evolution_agent.py
35     run_loop.sh
36     resume_run_loop.sh
```

第二阶段会牵涉大量 import 路径调整，必须配套完整回归测试。除非第一阶段 API 已稳定，否则不建议直接跳到第二阶段。

5. Public API 设计

5.1 包级导出

调用方只应从 SDK 包根入口导入稳定对象：

代码块

```
1  from qa_evolution_sdk import (
2      AgentConfig,
3      AgentTask,
4      AgentRunResult,
5      QuestionEvolutionAgent,
6      RunOptions,
7      ResumeOptions,
8      ReviewOptions,
9  )
```

`agent_runtime`、`pipeline_io`、`operator_router`、`scoring` 等仍是内部模块。调用方可以在高级场景中使用，但不承诺稳定。

5.2 客户端类

建议对外主类命名为 `QuestionEvolutionAgent`。如果后续需要区分“Agent 控制层”和“纯客户端”，可以增加别名 `QuestionEvolutionClient`。

代码块

```
1  class QuestionEvolutionAgent:
2      def __init__(
3          self,
4          *,
5          project_root: str | Path = ".",
6          config: AgentConfig | None = None,
7          runner_factory: RunnerFactory | None = None,
8      ) -> None:
9          ...
10
11     def dry_run(
12         self,
13         task: AgentTask | Mapping[str, Any],
14         *,
15         options: RunOptions | None = None,
16     ) -> AgentRunResult:
17         ...
18
19     def run(
20         self,
21         task: AgentTask | Mapping[str, Any],
22         *,
23         options: RunOptions | None = None,
24     ) -> AgentRunResult:
25         ...
26
27     def resume(
28         self,
29         task: AgentTask | Mapping[str, Any],
30         *,
31         options: ResumeOptions | None = None,
32     ) -> AgentRunResult:
33         ...
34
35     def review(
36         self,
37         *,
```

```
38         experiment_dir: str | Path,  
39         task: AgentTask | Mapping[str, Any] | None = None,  
40         options: ReviewOptions | None = None,  
41     ) -> AgentRunResult:  
42         ...
```

方法语义：

方法	作用	是否写实验产物
dry_run()	只生成 Agent 计划和上下文，不调用完整流水线。	写 Agent Run 产物，不写新实验产物。
run()	启动新实验，执行检查、完整流水线、观察、决策和报告。	写 Agent Run 产物和实验产物。
resume()	续跑已有实验，随后观察、决策和报告。	写 Agent Run 产物和实验续跑产物。
review()	只读复盘已有实验。	只写 Agent Run 报告，不修改实验正式产物。

5.3 快捷函数

可以提供少量快捷函数，方便脚本调用：

代码块

```
1  def run_question_evolution(  
2      task: AgentTask | Mapping[str, Any],  
3      *,  
4      project_root: str | Path = ".",  
5      config: AgentConfig | None = None,  
6      options: RunOptions | None = None,  
7  ) -> AgentRunResult:  
8      agent = QuestionEvolutionAgent(project_root=project_root, config=config)  
9      return agent.run(task, options=options)
```

快捷函数只能覆盖最常见场景，不应替代主类。

6. 输入对象设计

6.1 AgentTask

`AgentTask` 继续表示“要完成什么业务任务”。字段保持当前语义：

代码块

```
1 task = AgentTask(  
2     goal="找到 3 条边界候选",  
3     input_file="data/police_qa_testset_4scenes_sub.jsonl",  
4     search_mode="multi_operator_branch",  
5     boundary_target=3,  
6     max_search_steps=10,  
7 )
```

原则：

1. `AgentTask` 描述业务目标、输入和预算。
2. `AgentTask` 不保存 API key。
3. `AgentTask` 不保存运行时回调。
4. `AgentTask` 不保存 SDK 安装路径。

6.2 AgentConfig

`AgentConfig` 表示模型端点、并发、默认实验根目录和环境变量兼容策略。

代码块

```
1 config = AgentConfig(  
2     openai=ModelEndpointConfig(  
3         base_url="https://example.com/v1",  
4         api_key="...",  
5         model="gpt-5.4",  
6     ),  
7     qwen=ModelEndpointConfig(  
8         base_url="http://127.0.0.1:18011/v1",  
9         api_key="...",  
10        model="hjl_Qwen3.6-27B",  
11    ),  
12    scoring=ScoringConfig(  
13        request_timeout_seconds=180,  
14        retries=3,  
15        item_concurrency=20,  
16        qwen_answer_trials=3,  
17        qwen_judge_repeats=2,  
18        gpt_judge_repeats=2,  
19    ),
```



```
20     exp_root="experiments",
21 )
```

配置对象需要支持三种创建方式：

代码块

```
1  AgentConfig()
2  AgentConfig.from_env()
3  AgentConfig.from_mapping({...})
```

优先级建议：

代码块

```
1  显式传入的 AgentConfig
2  > options.env_overrides 中白名单字段
3  > 当前进程环境变量
4  > 项目默认值
```

6.3 RunOptions

`RunOptions` 表示这次调用的执行控制，不属于业务任务本身。

代码块

```
1  options = RunOptions(
2      agent_run_id="sdk-smoke-001",
3      progress=print,
4      stream_output=True,
5  )
```

建议字段：

字段	说明
agent_run_id	可选固定运行 ID，主要用于测试和可复现运行。
progress	进度回调，接收短文本。
event_callback	结构化事件回调，接收工具开始、完成、失败等事件摘要。
stream_output	是否把底层工具输出转发给回调。
env_overrides	仅允许白名单环境变量覆盖。
timeout_seconds	SDK 外层保护超时，不能替代工具级 timeout。

`ResumeOptions` 和 `ReviewOptions` 可以先继承或复用 `RunOptions`，只有在有明确差异时新增字段。

7. 返回对象设计

7.1 AgentRunResult

当前 `execute_agent_task()` 返回 dict。SDK 对外应返回稳定对象：

代码块

```
1  @dataclass(frozen=True)
2  class AgentRunResult:
3      state: Mapping[str, Any]
4      plan: Mapping[str, Any]
5      observation: Mapping[str, Any] | None = None
6      decision: Mapping[str, Any] | None = None
7      tool_results: tuple[Mapping[str, Any], ...] = ()
8
9      @property
10     def status(self) -> str: ...
11
12     @property
13     def agent_run_dir(self) -> Path: ...
14
15     @property
```

```

16     def experiment_dir(self) -> Path | None: ...
17
18     @property
19     def report_path(self) -> Path | None: ...
20
21     def to_dict(self) -> dict[str, Any]: ...

```

调用方常用读取方式：

代码块

```

1  result = agent.run(task)
2
3  print(result.status)
4  print(result.experiment_dir)
5  print(result.report_path)

```

7.2 状态语义

SDK 不应把所有非 completed 状态都当成异常。

状态	SDK 行为
completed	正常返回。
suspended	正常返回，调用方根据 requires_manual_review 或 terminal_reason 决定后续动作。
blocked	正常返回或抛异常应可配置；默认建议正常返回，保留完整报告。
输入非法	抛出 InvalidTaskError 或 PolicyViolationError。
SDK 自身无法创建运行目录	抛出 SDKRuntimeError。

这样设计的原因是：在当前项目语义里，`manual_review_required`、`blocked`、`score_increased` 等很多结果是实验事实或治理结果，不一定是 Python 调用失败。

8. 异常设计

建议新增 SDK 异常层：

代码块

```
1 class QuestionEvolutionSDKError(Exception): ...
2 class InvalidTaskError(QuestionEvolutionSDKError): ...
3 class InvalidConfigError(QuestionEvolutionSDKError): ...
4 class PolicyViolationError(QuestionEvolutionSDKError): ...
5 class SDKRuntimeError(QuestionEvolutionSDKError): ...
6 class ArtifactError(QuestionEvolutionSDKError): ...
```

映射规则：

内部错误	SDK 异常
AgentTaskError	InvalidTaskError
PolicyViolation	PolicyViolationError
配置字段非法	InvalidConfigError
运行目录创建失败	SDKRuntimeError
产物读取或转换失败	ArtifactError

工具执行失败默认不直接抛异常，而是进入 AgentRunResult。如果调用方设置 raise_on_blocked=True，可以在结果生成后抛出带 result 的异常。

9. 配置到环境变量的转换

第一版仍调用现有复合工具，因此需要把结构化配置转换成受控环境变量。

转换原则：

- 1. 只转换白名单字段。
- 2. 不在事件日志中写完整 API key。
- 3. 明确区分 SDK 配置和用户当前 shell 环境。
- 4. 显式传入配置优先于环境变量。
- 5. 不允许调用方传任意环境变量绕过 Policy Guard。

示例映射：

SDK 字段	环境变量
task.input_file	INPUT_FILE
config.exp_root	EXP_ROOT
task.search_mode	SEARCH_MODE
task.boundary_target	BOUNDARY_TARGET
task.max_search_steps	MAX_SEARCH_STEPS
task.execution_scope	EXECUTION_SCOPE
config.scoring.request_timeout_seconds	SCORING_REQUEST_TIMEOUT_SECONDS
config.scoring.retries	SCORING_RETRIES
config.scoring.item_concurrency	SCORING_ITEM_CONCURRENCY
config.scoring.qwen_answer_trials	QWEN_ANSWER_TRIALS

模型端点和 key 的处理建议分两阶段：

1. 第一阶段：继续由现有环境变量读取模型端点和 key，SDK 只提供 `AgentConfig.to_env()` 方便注入。
2. 第二阶段：把各阶段模型客户端改成显式依赖注入，减少全局环境变量依赖。

10. CLI 兼容策略

现有 CLI 命令保持不变：

代码块

```
1 python question_evolution_agent.py --project-root . dry-run --task task.json
2 python question_evolution_agent.py --project-root . run --task task.json
3 python question_evolution_agent.py --project-root . resume --task task.json
4 python question_evolution_agent.py --project-root . review --exp-dir
  experiments/...
```

改造后 CLI 内部调用 SDK：

代码块

```
1  parse args
2  -> load AgentTask
3  -> QuestionEvolutionAgent(project_root=...)
4  -> dry_run / run / resume / review
5  -> print result.to_dict() as JSON
```

CLI 保持 stdout 输出 JSON，progress 继续写 stderr。这样已有测试和自动化脚本可以继续工作。

11. 示例用法

11.1 新实验

代码块

```
1  from qa_evolution_sdk import AgentConfig, AgentTask, QuestionEvolutionAgent
2
3  agent = QuestionEvolutionAgent(
4      project_root=".",
5      config=AgentConfig.from_env(),
6  )
7
8  task = AgentTask(
9      goal="找到 3 条边界候选",
10     input_file="data/police_qa_testset_4scenes_sub.jsonl",
11     search_mode="multi_operator_branch",
12     boundary_target=3,
13     max_search_steps=10,
14 )
15
16 result = agent.run(task)
17
18 print(result.status)
19 print(result.experiment_dir)
20 print(result.report_path)
```

11.2 续跑实验

代码块

```
1  from qa_evolution_sdk import AgentTask, QuestionEvolutionAgent
2
3  agent = QuestionEvolutionAgent(project_root=".")
4
5  task = AgentTask(
6      goal="续跑已有实验",
```

```
7     resume_exp_dir="experiments/2026-08-07/exp",
8     resume_start_round=2,
9 )
10
11 result = agent.resume(task)
```

11.3 只读复盘

代码块

```
1 from qa_evolution_sdk import QuestionEvolutionAgent
2
3 agent = QuestionEvolutionAgent(project_root=".")
4
5 result = agent.review(
6     experiment_dir="experiments/2026-08-07/exp",
7 )
8
9 print(result.report_path)
```

11.4 测试注入 Fake Runner

代码块

```
1 agent = QuestionEvolutionAgent(
2     project_root=tmp_path,
3     runner_factory=lambda root, logger: FakeRunner(root, logger),
4 )
5
6 result = agent.run(task, options=RunOptions(agent_run_id="sdk-test"))
```

这能保留当前测试风格，不需要真实模型调用。

12. 分阶段实施计划

12.1 阶段 A：新增 SDK 门面

新增：

代码块

```
1 qa_evolution_sdk/__init__.py
2 qa_evolution_sdk/client.py
3 qa_evolution_sdk/results.py
```

实现内容：

1. `QuestionEvolutionAgent` 包装现有 `execute_agent_task()`。
2. `AgentRunResult` 从现有 dict 构造。
3. 包根导出 `AgentTask`、`QuestionEvolutionAgent`、`AgentRunResult`。
4. 新增 SDK client 测试，使用 fake runner。

验收：

1. `from qa_evolution_sdk import QuestionEvolutionAgent` 可用。
2. `agent.dry_run(task)` 可生成 plan。
3. `agent.run(task)` 在 fake runner 下通过。
4. 现有 CLI 测试不变。

12.2 阶段 B：新增配置与选项对象

新增：

代码块

```
1 qa_evolution_sdk/config.py
2 qa_evolution_sdk/options.py
3 qa_evolution_sdk/callbacks.py
```

实现内容：

1. `AgentConfig.from_env()`。
2. `AgentConfig.to_env_overrides()`。
3. `RunOptions`、`ResumeOptions`、`ReviewOptions`。
4. progress callback 和 event callback 协议。
5. 配置字段校验和敏感字段脱敏。

验收：

1. 显式 `AgentConfig` 能覆盖默认实验根目录和评分并发。
2. 非白名单 env override 被拒绝。
3. API key 不进入事件摘要。
4. fake runner 测试不访问真实 API。

12.3 阶段 C：CLI 改成 SDK 薄封装

修改：

代码块

```
1 question_evolution_agent.py
```

实现内容：

1. CLI 只负责参数解析和 JSON 输出。
2. 业务执行统一调用 `QuestionEvolutionAgent`。
3. 旧入口函数保留兼容导入，必要时标记为 deprecated。

验收：

1. 旧 CLI 命令行为不变。
2. stdout 仍是 JSON。
3. stderr 仍输出进度。
4. 现有 `test_agent_cli.py` 通过。

12.4 阶段 D：标准打包

新增：

代码块

```
1 pyproject.toml
```

实现内容：

1. 定义包名、Python 版本、依赖和可选测试依赖。
2. 配置 console script，例如 `qa-evolution-agent`。
3. 确保 schema、prompts、memory 默认文件能被包访问或明确仍走项目根路径。

验收：

1. `pip install -e .` 成功。
2. `qa-evolution-agent --help` 可用。
3. import 测试通过。
4. 不改变现有 `python question_evolution_agent.py ...` 用法。

12.5 阶段 E：减少 Shell 依赖

这一阶段不是第一版 SDK 必需项。

目标：

1. 把 `run_loop.sh` 中的阶段编排逐步迁移为 Python orchestration。
2. `ToolRunner` 可以选择 `SubprocessRunner` 或 `PythonRunner`。
3. SDK 调用时减少对 shell、cwd 和环境变量的依赖。

验收：

1. Python runner 与 shell runner 产物兼容。
2. manifest、断点恢复、summary 和 final 输出一致。
3. 旧 shell 入口仍保留一段兼容期。

13. 兼容与弃用策略

必须兼容：

1. 现有 `question_evolution_agent.py` CLI。
2. 现有 `run_loop.sh` 和 `resume_run_loop.sh`。
3. 现有 `AgentTask` JSON。
4. 现有 `agent_runs` 产物布局。
5. 现有实验目录布局。
6. 现有测试中 fake runner 注入方式。

可以弃用但不能立刻删除：

1. 直接 import `execute_agent_task()` 作为外部入口。
2. 直接从 CLI 文件 import `main()` 以外的内部 helper。
3. 通过任意环境变量控制执行的方式。

弃用建议：

代码块

- 1 第一版：文档中标记推荐使用 `qa_evolution_sdk`。
- 2 第二版：对旧 import 路径发 `DeprecationWarning`。
- 3 第三版：只保留 CLI 和 SDK 公共入口，内部 helper 不承诺兼容。

14. 测试计划

新增测试：

测试	目的
test_sdk_client_dry_run	验证 SDK 能直接生成计划。
test_sdk_client_run_with_fake_runner	验证 SDK 能执行新实验流程。
test_sdk_resume_with_fake_runner	验证续跑入口。
test_sdk_review_existing_experiment	验证只读复盘不修改实验产物。
test_sdk_config_from_env	验证配置读取和默认值。
test_sdk_config_rejects_unknown_env_override	验证 Policy Guard 不被绕过。
test_sdk_result_properties	验证 status、experiment_dir、report_path 等便捷属性。
test_cli_uses_sdk_and_keeps_stdout_json	验证 CLI 兼容。

必须继续运行：

代码块

```
1  pytest -q
2  bash -n run_loop.sh
3  bash -n resume_run_loop.sh
```

如果只改 SDK 门面，最低验收可以先跑：

代码块

```
1  pytest -q tests/test_agent_cli.py tests/test_sdk_client.py
```

15. 风险与处理

风险	表现	处理
import 路径混乱	CLI、测试和 SDK 各走一套路径。	第一阶段不移动现有 runtime，只新增门面。
配置重复	env、 config.py 、AgentConfig 三处都能配置。	明确优先级，并在结果中记录有效配置摘要。
API key 泄露	事件、报告或异常里出现完整 key。	配置对象统一脱敏，事件层只记录字段名和是否存在。
SDK 绕过治理	调用方直接传入未登记工具或任意 env。	所有调用仍走 Policy Guard 和 Tool Registry。
返回状态误解	调用方把 suspended 当作 Python 异常。	文档明确状态语义，默认返回结构化结果。
一次性重构过大	改包结构同时改业务流水线。	先做门面，再做配置，再做打包，最后去 Shell 化。

16. Reader Test

按“新工程师读完后能否开始实施 SDK 化”的标准检查：

- 1. 能看出第一版 SDK 的边界：只包现有 Agent Harness，不重写业务流水线。
- 2. 能知道新增哪些目录和文件。
- 3. 能知道公共 API 包括哪些类、方法、输入、返回和异常。
- 4. 能知道配置如何从对象转成现有环境变量。
- 5. 能知道 CLI 如何保持兼容。
- 6. 能按阶段 A 到 E 逐步落地，并用测试验证每一步。

仍需实现前确认的一点：最终包名使用 `qa_evolution_sdk` 还是 `qa_evolution`。本文建议第一阶段先用 `qa_evolution_sdk`，因为它改动小、不会和现有根目录脚本命名冲突；第二阶段如做整体包整理，再收敛到 `qa_evolution`。